

— ZERO TRAINING · NO TASK HEAD · OPEN VOCABULARY

Test-time compute of a frozen jina-v5-omni for image tagging

Han Xiao

VP of AI, Elastic

[@hxiao](#) · [in/hxiao87](#)

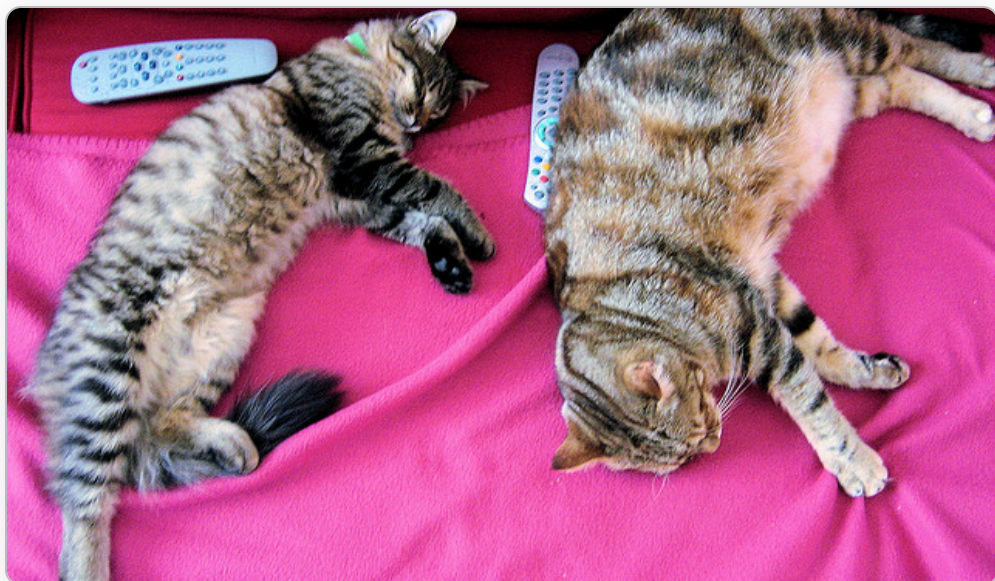
GITHUB REPO



[code](#) · [benchmark](#) · [findings](#)

— THE RESULT, FIRST

A frozen retrieval model that tags: zero training, open vocabulary.



cat.jpg, 86 ms → [kitty](#), [cosy](#), [plush](#), [crib](#), [paw](#), [sleeps](#)

0.813

precision@1 on COCO-150 multi-label, 14-crop mode (fast mode: 0.753 at 75 ms)

128,260

tokens in the label vocabulary, self-gated to 25,465 words at scoring time: no curated tag list

0

training steps, second models, dictionaries, annotations: everything is test-time compute

Deployed on-device over images, video, and scanned documents. The rest of this talk: [where this capability comes from, and which test-time compute pays for it.](#)

Three questions prior work leaves open.

Prior tagging work requires training, a curated label list, or external lexical resources. This talk removes all three:

RQ1 Can the tokenizer vocabulary replace a curated label set?

RAM / RAM++ train over ~6,400 curated tags; TagCLIP (AAAI'24) and PIAA ("[CLS] is not enough") assume a given class list.

RQ2 Can one frozen model supply every signal: features, labels, word filter, dedup?

Prior training-free work: two-tower CLIP plus WordNet, dictionaries, POS tags.

RQ3 Which test-time compute scales accuracy: re-processing features, or new information?

The literature bets on re-processing: whitening (PIAA), propagation (ZLaP), transport (OTTER), priors (BCA). Measured here against re-encoding new crops.

Test-time compute for an embedding model: three families.

Language models scale test-time compute in tokens: longer chains, more samples. A frozen encoder has no tokens to spend. Its inference-time budget takes exactly three forms:

A MORE COMPUTATION PER PASS

Extract more from a single forward pass: score every patch, fuse channels, subtract priors. Cost: a few matrix multiplies.

LLM analogue: a richer readout of one sample.

B NEW PASSES, NEW VIEWS

Run the frozen encoder **again** on inputs it has not seen: crops of the image, splits of the document. Cost: full forward passes.

LLM analogue: best-of-N sampling.

C RE-PROCESS THE FEATURES

Transform the vectors already computed: whitening, graph propagation, optimal transport, re-normalization. Cost: nearly zero.

Where most of the 2024–2026 training-free literature operates.

This talk is a controlled experiment across all three families.
Two of them scale accuracy with compute. One does not.

Open-vocabulary, multi-label tagging, under a strict constraint.

jina-embeddings-v5-omni-nano is a ~1B retrieval encoder: no tagging head, no classifier, no tag list. It was never trained to answer “what is in this image”. The task: given an image, return the words for **every** object in it, from an open vocabulary.

- | **OPEN-VOCABULARY** no curated tag list; the label space is the model’s own.
- | **MULTI-LABEL** a photo has many objects; recall them all.
- | **SINGLE FROZEN MODEL** every signal from one encoder, or the model is no longer doing the work.

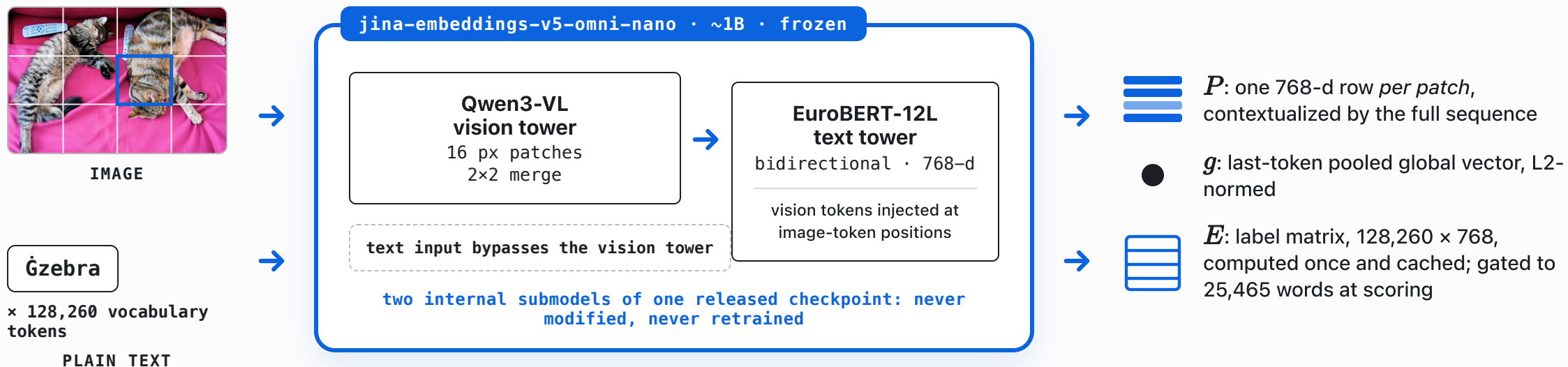
ZERO TRAINING

NO SECOND MODEL

NO WORDNET / DICTIONARY

NO REGEX / POS TAGGER

One frozen checkpoint, two input paths, one 768-d space.



Both paths end in the same space, so $\langle p, e_\ell \rangle$ is well-defined: **any patch of any image, scored against any word the model knows.**

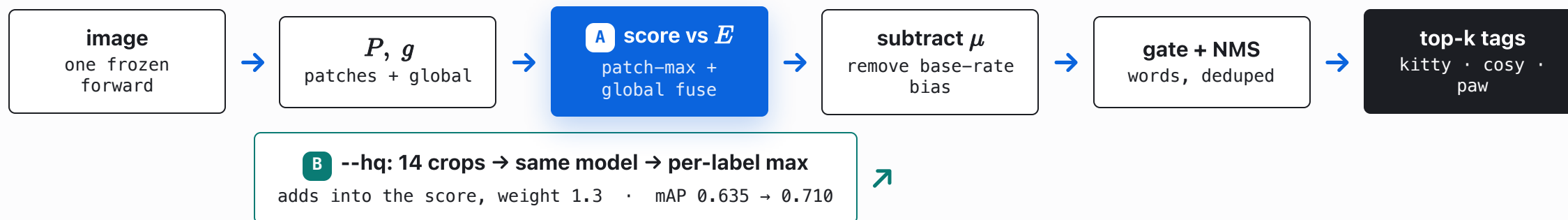
— THE PIPELINE

The complete tagger: test-time compute around a frozen forward pass.

OFFLINE · COMPUTED ONCE, CACHED



PER IMAGE · 75 MS FAST · 1016 MS --HQ



Every box is the frozen model or plain arithmetic. **A** is more computation on one pass; **B** is new passes on new pixels. No box is family C.

The label set is the tokenizer's own vocabulary.

No curated tag list. Take all **128,260** tokens and encode each one as `text` into a label matrix

$$E = [e_\ell]_{\ell \in V}, \quad e_\ell = \text{encode_text}(\ell) \in \mathbb{R}^{768}, \quad |V| = 128,260$$

One matmul now scores any image against the entire vocabulary; fragments are filtered later by a word-start gate (step 4).

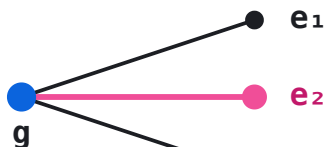
```
wrong: image_vec · embed_tokens.weight  
       → Tutor, avatar, PyTuple (garbage)  
right: image_vec · encode_text(token)  
       → kitty, cosy, plush, paw (aligned)
```

Why `encode_text`, not the embedding table? The model has `tie_word_embeddings = false`: the input-embedding table lives in a different space than the pooled output. You must encode each token as a *text string* to put labels in the image's space.

The original intuition (use the vocabulary directly) was correct, but only through the aligned output space.

One pooled vector under-detects; score every patch instead.

GLOBAL POOLED

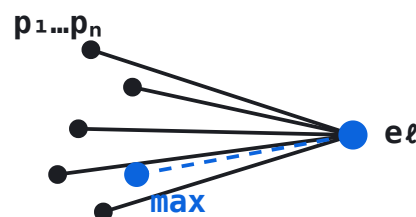


$$s_\ell = \langle g, e_\ell \rangle$$

one vector dominated by the most salient object

mAP 0.264

PATCH MAX + GLOBAL

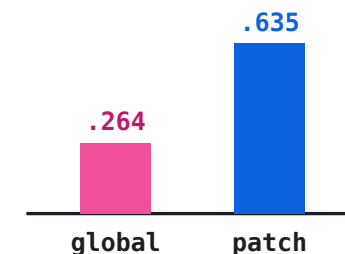


$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \langle g, e_\ell \rangle$$

a small object fires on its own patch and survives

mAP 0.635

THE GAIN



+0.37 mAP

PIAA's claim, "[CLS] is not enough", confirmed on this encoder

SAME PIXELS, MORE COMPUTATION

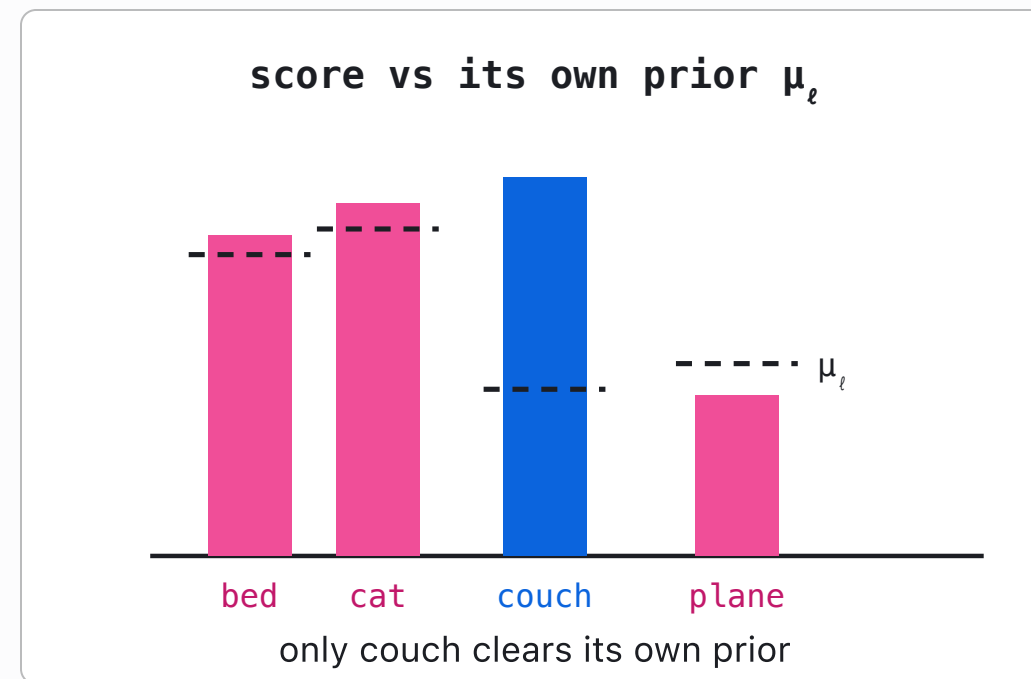
A Pure family-A compute: the patch features already exist in the forward pass. One additional max over rows yields +0.37 mAP.

Generic words sit close to every image: subtract the per-label prior.

Raw cosine has a base-rate bias: generic words ("bed", "cat") score high on almost any image because they sit near the image modality. Estimate a per-label prior once from neutral background images, then subtract it:

$$\tilde{s}_\ell = s_\ell - \mu_\ell, \quad \mu_\ell = \mathbb{E}_{x \sim \text{background}} [s_\ell(x)]$$

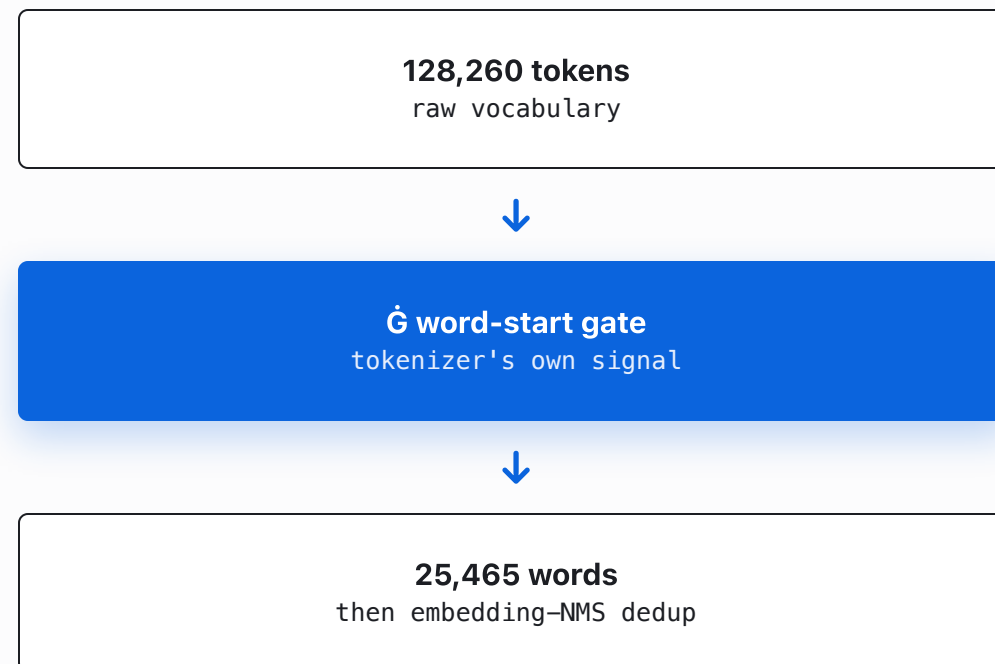
No calibration set, no labels, one offline pass: only image-specific spikes survive.



The tokenizer already knows what a word is.

Word-start gate. Byte-BPE marks whole-word starts with a space glyph. `Ġcat` is a word; `GetComponent` is a fragment. That one built-in signal filters 128k tokens down to **25,465** clean words. No external dictionary.

Embedding-NMS. Synonyms and multilingual variants (`кот` / `Cat` / `кот` / `cat`) collapse into one, using cosine in the model's own space. Walk labels by score; keep ℓ only if $\max_{k \in \text{kept}} \langle e_\ell, e_k \rangle < 0.6$. Again: no lookup table, just the geometry.



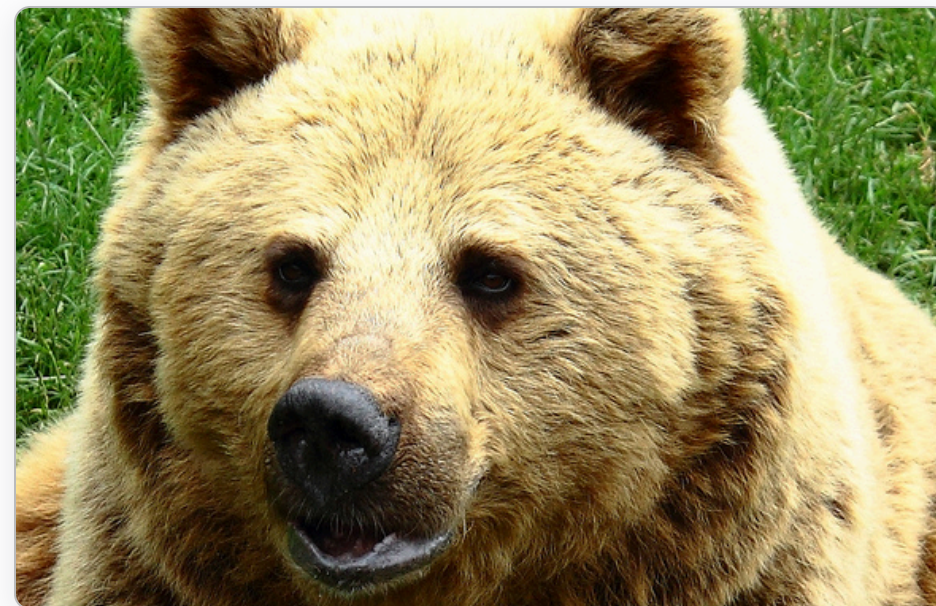
CWR multi-crop: the one lever that raises the accuracy ceiling.

Class-Wise Reranking (CWR), the crop-and-rescore idea from TagCLIP (AAAI'24), extended to a full-coverage **3×3 + 2×2 + center** grid: 14 crops through the same frozen model, per-label **max** across crops. A small object fills whichever crop contains it: weak signal becomes strong.



$$S_\ell = \tilde{s}_\ell + 1.3 \left(\max_{c=1..14} s_{c,\ell} - \mu_\ell \right), \quad s_{c,\ell} = \max \left(\max_{p \in P_c} \langle p, e_\ell \rangle, \langle g_c, e_\ell \rangle \right)$$

B Per-label **max**, not averaging: **averaging hurt**. The outlier crop is the evidence.



fast: wolf, roar, muzzle → --hq: fur, bear, muzzle
multi-crop re-encoding corrects the single-pass misreading

The entire tagger is one scoring function.

$$S(\ell) = \underbrace{0.7 \left(\max_{p \in P} \langle p, e_\ell \rangle - \mu_\ell \right)}_{\text{patch evidence}} + \underbrace{0.3 \left(\langle g, e_\ell \rangle - \mu_\ell^g \right)}_{\text{global context}} + \underbrace{1.3 \left(\max_{c=1..14} s_{c,\ell} - \mu_\ell \right)}_{\text{multi-crop re-encode}}$$

A Patch evidence: algebra over rows the forward pass already produced. Free. +0.37 mAP.

A Global context: the pooled vector, de-biased by its own background prior. Free.

B Multi-crop: 14 fresh forward passes on new pixels. 14× the cost. +0.075 mAP.

then $\ell \in \hat{G}$ -gated words $\rightarrow$ embedding-NMS ($\tau = 0.6$) $\rightarrow$ top- k

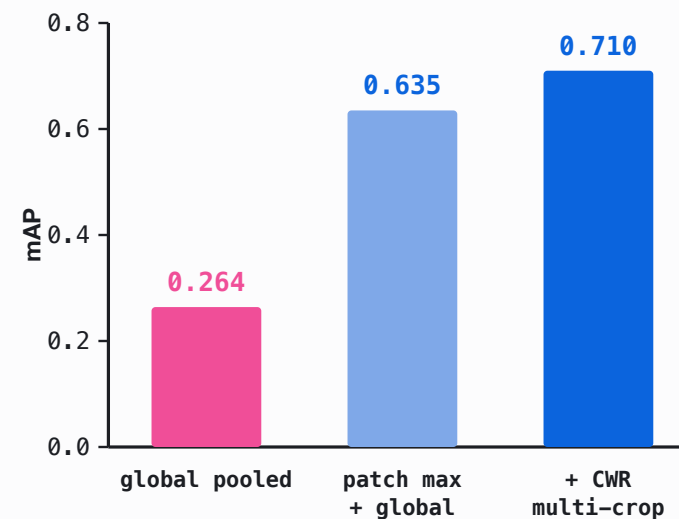
No head, no logits, no learned threshold: **three fixed weights, two background priors, and max operations over frozen-model outputs.**

RESULTS

COCO-150, 80 categories: the full progression.

METHOD	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

150 COCO-val images, real multi-label ground truth, avg 2.93 labels/image. softpool trades mAP for top-k precision; the ladder tracks the max-pool path.



mAP: global → patch → +CWR

Open vocabulary, out-of-distribution photos.



AI conference hall · --hq

onstage

attendees

ceilings

seats



Porsche 911 interior · --hq

carro

leather

steering

clock

seat



bear on grass · --hq

fur

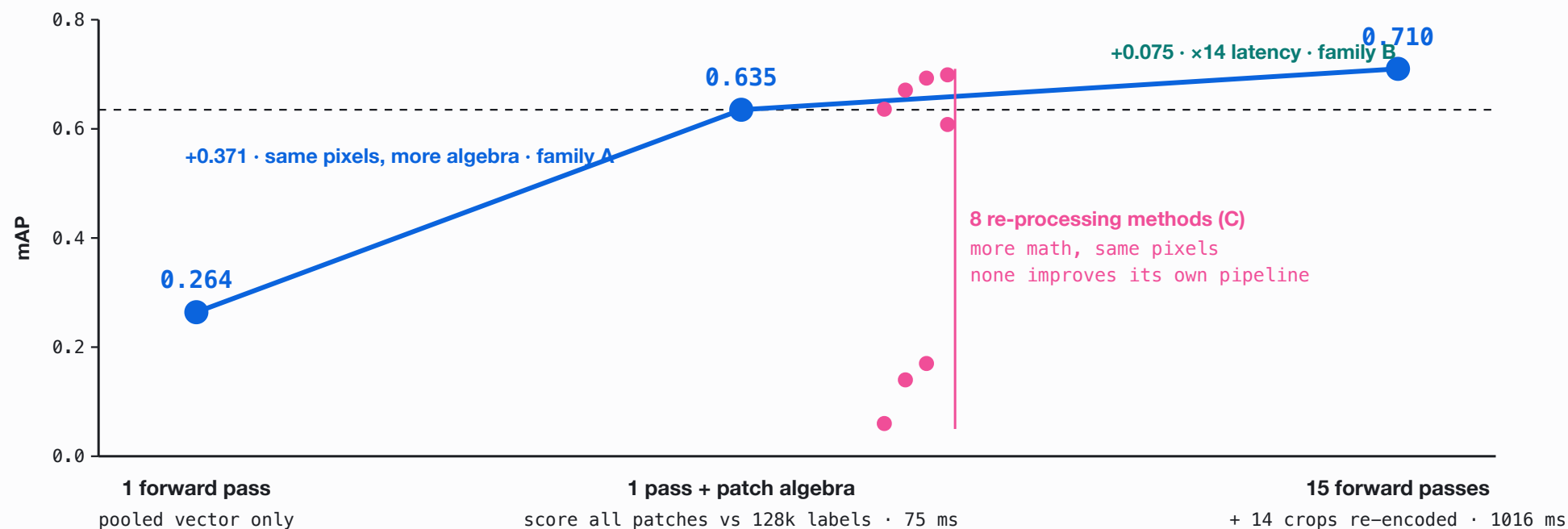
bear

muzzle

ivory

Labels come directly from the tokenizer, so multilingual variants ("carro") and occasional fragments appear. That is the direct cost of a **zero-annotation** open vocabulary.

Accuracy as a function of test-time compute.

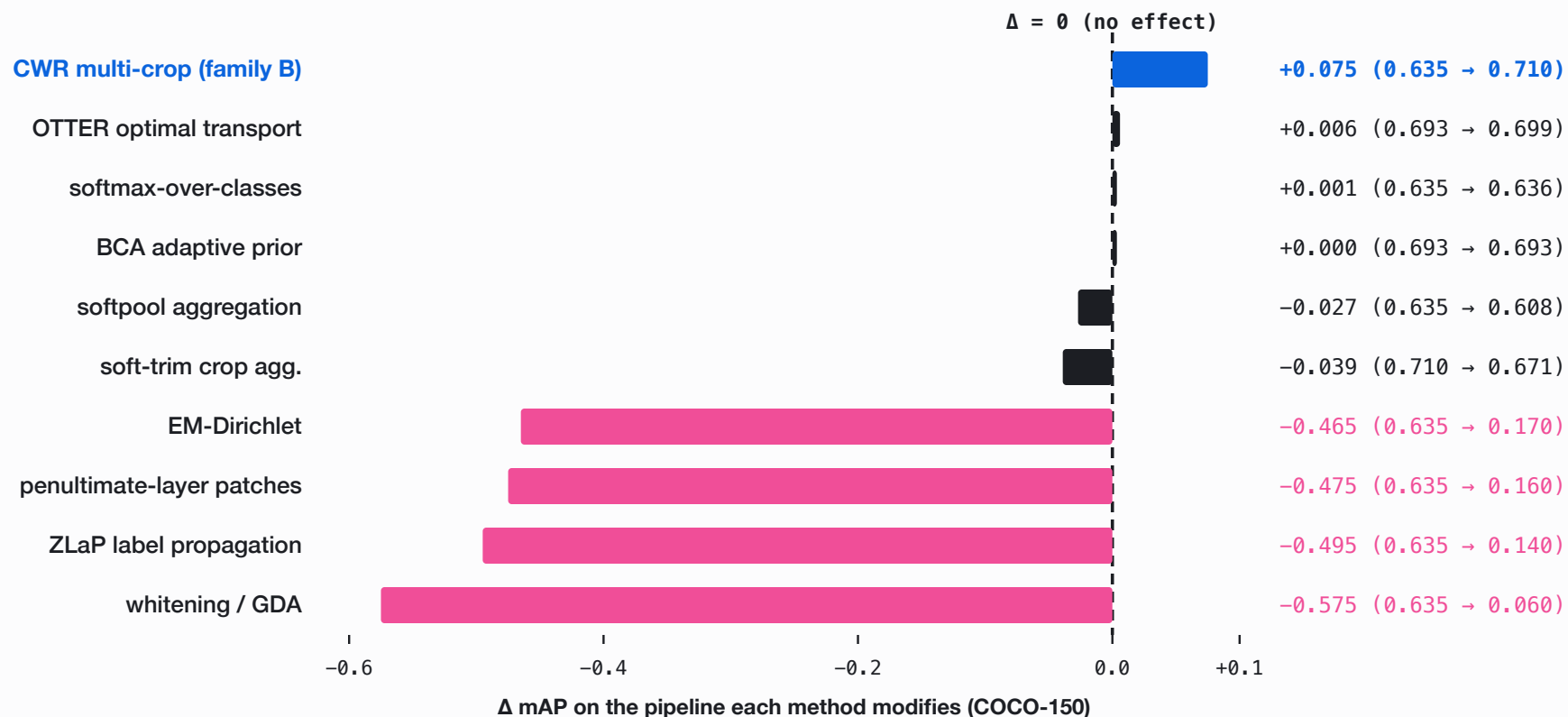


Accuracy scales along exactly one axis: **the compute that carries new information**. Family A is nearly free; family B pays 14× latency for +0.075. The pink cluster is the subject of the next slide.

THE EXPERIMENT

Ten training-free levers; only re-encoding adds accuracy.

RQ3, answered by measurement: every published lever, re-implemented. Each bar: Δ mAP on the pipeline the method modifies.



Re-processing well-calibrated features yields nothing; new information does.

Re-processing methods were designed for weakly calibrated, single-label, two-tower CLIP. This encoder's space is already aligned, calibrated, and multi-label: **there is nothing left to recover.**

RE-PROCESSING FEATURES

whitening 0.06 · ZLaP 0.14 · EM-Dirichlet 0.17 · OTTER +0.006
over its own base. None improves the pipeline it modifies.

FEEDING NEW INFORMATION

CWR crops → mAP 0.710, P@1 0.813. The only intervention that improves results.

The ceiling is the 1B model itself. Real test-time compute here means **giving the model more to look at**, not re-arranging what it already saw.

Versus the recent tagging literature.

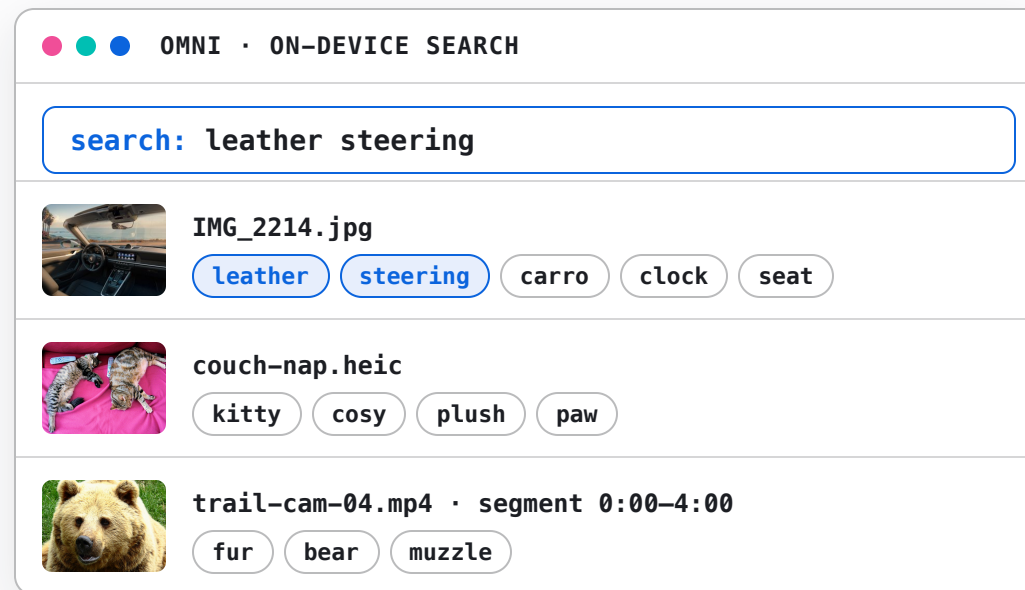
LINE OF WORK	THEIR SETUP	OUR DELTA
Tag2Text / RAM / RAM++	trained tagging models, curated ~6.4k tag list	zero training, label space = tokenizer vocab
TagCLIP	CLIP two-tower, penultimate layer, DMAR+CWR	single omni model, last layer is the output space
PIAA	patch-level scoring ("[CLS] is not enough") + GDA whitening	patch thesis confirmed (+0.37 mAP); whitening collapses (0.06)
recent calibration (OTTER, BCA)	calibration set or target prior to de-bias	one offline background prior, zero calibration

RQ1: the vocabulary suffices as the label space. **RQ2:** one frozen model supplies every signal. **RQ3:** only new-information compute scales. Three questions, answered.

From study to production: on-device tagging in Omni.

The Python study became a feature of **Omni**, a native on-device semantic-search app (Swift + MLX, same frozen model). The port reuses the architecture directly:

- | SAME FORWARD PASS the patch rows already exist for the file's *embedding*; tagging adds one matmul.
- | ~0.6 MS / IMAGE about 4% overhead on the embed step. Tagging is effectively free at index time.
- | TAGS = SNIPPETS tags land in the search snippet: media becomes *keyword*-searchable.
- | SELF-CALIBRATING the background prior μ is estimated on-device from the first 64 images it sees, then frozen.



tag chips as they appear in results; the third row is a video segment

One label matrix: images, HQ re-encoding, video, and scanned documents.



IMAGE

Tagged at **index time**, inside the embedding forward. Patch-max + global, prior-centered, gated, NMS-deduped.

+1 matmul · ~4% overhead



HQ IMAGE

A background **re-tag pass** adds the production CWR variant: 5 crops, per-label max.

P@1 0.773 → 0.847 (Swift, bf16) · 14-crop config: not deployed



VIDEO

32 frames per **240-second segment**, one sequence through the tower: patch-max pools over **space and time**.

one tag set per segment



SCANNED PDF

Pages render to images, same path: **per-page tags** ("invoice, table, signature") as the snippet.

every page, keyword-findable

One label matrix, one scoring rule. The media shape only changes **what counts as a patch**: crops for detail, frames for time, pages for documents.

A capability the model was never trained for, realized at inference.

Two talks, one thesis. A frozen encoder holds more capability than its training objective exposes, and you unlock it with **test-time compute**, not more parameters.

RETRIEVAL TALK

recombine the geometry for more **relevance** on the task it was trained for.

THIS TALK

recombine the geometry for an **emergent new task** it was never trained for: tagging.

Test-time compute scales only when it **supplies the model with new information**. Re-processing an already-good representation yields nothing.

**A frozen model already knows more
than its objective admits.**
Test-time compute is how you ask.

Han Xiao · VP of AI, Elastic

[@hxiao](#) · [in/hxiao87](#)

github.com/hanxiao/jina-v5-omni-nano-test-time-image-tagging

GITHUB REPO



[code](#) · [benchmark](#) · [findings](#)